

Audio Fingerprinting and Song Identification using Spectrogram Peak Hashing

[Live Demo](#) [GitHub Repository](#)

Aarush Saharan, Advika Goel

June 22, 2026

Abstract

This project implements a simplified audio fingerprinting system for song identification using spectrogram peak hashing. The audio signal is analyzed using the Short-Time Fourier Transform (STFT) to generate a spectrogram, from which prominent spectral peaks are extracted to form a constellation map. Pairs of peaks are then combined to create compact fingerprints that are stored in a database. Song identification is performed by matching fingerprints generated from a query clip against those stored in the database. The performance of the system is evaluated using clips from the same song and different songs. Additionally, robustness experiments are conducted by introducing additive noise, pitch shifts, and time stretching. The results demonstrate that the system can successfully identify songs under normal conditions, while also highlighting the trade-offs between robustness and discrimination when modifications are introduced.

1 Introduction

1.1 Background

Audio fingerprinting is a technique used to identify an audio recording from a short sample by extracting a compact set of distinctive features called fingerprints. Instead of comparing entire audio files, the method stores only the most characteristic information, making identification efficient and scalable. Since music signals change continuously over time, time-frequency analysis methods such as the Short-Time Fourier Transform (STFT) are used to generate spectrograms that capture how frequency content evolves. Modern systems such as Shazam rely on prominent spectral peaks and their relationships to create robust fingerprints. In this project, a simplified audio fingerprinting system based on spectrogram peak hashing is implemented to identify songs and study the effects of noise, pitch shifting, and time stretching on recognition performance.

1.2 Objective of the Project

The objective of this project is to implement a basic audio fingerprinting system capable of identifying a song from a short query clip. The system uses the Short-Time Fourier Transform (STFT) to generate a spectrogram, extracts prominent spectral peaks, and creates fingerprints using pairs of peaks. These fingerprints are stored in a database and later used for song identification. In addition, the robustness of the system is evaluated by studying the effects of additive noise, pitch shifting, and time stretching on identification accuracy.

1.3 Overview of the Fingerprinting System

The implemented fingerprinting system consists of five main stages. First, the input audio signal is converted into a spectrogram using the Short-Time Fourier Transform (STFT). Next,

prominent spectral peaks are detected and represented as a constellation map in the time-frequency plane. These peaks are then paired to generate fingerprints in the form of hashes that capture both frequency and timing information. The generated fingerprints are stored in a database for known songs. Finally, fingerprints extracted from a query clip are compared against the database, and the number of matching hashes is used to determine whether the query corresponds to a stored song.

2 Theory

2.1 Fourier Transform

The Fourier Transform is a mathematical tool used to analyze the frequency content of a signal. It decomposes a time-domain signal into a sum of sinusoidal components of different frequencies, allowing the amplitudes of these frequencies to be examined. For a continuous-time signal $x(t)$, the Fourier Transform is defined as

$$X(f) = \int_{-\infty}^{\infty} x(t) e^{-j2\pi ft} dt.$$

While the Fourier Transform provides information about which frequencies are present in a signal, it does not indicate when those frequencies occur. This limitation makes it unsuitable for analyzing music and speech signals, whose frequency content changes continuously over time. Consequently, a time-frequency analysis technique such as the Short-Time Fourier Transform (STFT) is required to capture both temporal and spectral information.

2.2 Short-Time Fourier Transform (STFT)

To analyze signals whose frequency content changes with time, the Short-Time Fourier Transform (STFT) is used. Instead of applying a single Fourier Transform to the entire signal, the STFT divides the signal into short overlapping segments called windows and computes a Fourier Transform for each segment separately. This produces a sequence of frequency spectra that describe how the signal evolves over time.

For a discrete-time signal $x[n]$, the STFT is given by

$$X(m, k) = \sum_{n=-\infty}^{\infty} x[n] w[n - m] e^{-j2\pi kn/N},$$

where $w[n]$ is the window function, N is the FFT length, and m determines the position of the window in time. The magnitude of the STFT can be visualized as a spectrogram, which displays time on the horizontal axis, frequency on the vertical axis, and signal intensity through color. The STFT therefore provides both temporal and spectral information, making it particularly suitable for analyzing music signals and forming the basis of the audio fingerprinting system implemented in this project.

2.3 Spectrograms

A spectrogram is a visual representation of the frequency content of a signal as it changes over time. It is obtained by computing the magnitude of the Short-Time Fourier Transform (STFT) for successive overlapping windows of the signal. In a spectrogram, the horizontal axis represents time, the vertical axis represents frequency, and the color intensity represents the magnitude of the corresponding frequency component.

The choice of window length plays an important role in spectrogram analysis. A short window provides good time resolution, allowing rapid changes in the signal to be detected, but

results in poor frequency resolution. Conversely, a long window improves frequency resolution but reduces the ability to localize events in time. This trade-off between time and frequency resolution is a fundamental characteristic of the STFT. In this project, the spectrogram serves as the primary representation of the audio signal from which prominent spectral peaks are extracted for fingerprint generation.

2.4 Audio Fingerprinting

Audio fingerprinting is the process of generating a compact and distinctive representation of an audio recording that can be used for identification. Rather than storing the complete audio waveform, the system extracts a set of salient features that capture the unique characteristics of the signal. These features are then converted into fingerprints, which can be efficiently stored and compared within a database.

A desirable fingerprint should satisfy three important properties: uniqueness, compactness, and robustness. It should uniquely distinguish one song from another, require relatively little storage space, and remain recognizable even when the audio is affected by noise or other minor distortions. In this project, fingerprints are generated from prominent peaks in the spectrogram. Pairs of peaks are combined into hashes containing frequency and timing information, allowing songs to be identified by matching these hashes against a database of known recordings.

3 Methodology

3.1 Audio Loading and Preprocessing

The first step in the fingerprinting pipeline is to load the audio file and convert it into a digital signal suitable for processing. In this project, audio files in MP3 format were loaded using the `librosa` library. The audio signal was represented as a one-dimensional array containing the amplitude of the waveform at discrete time instants, while the sampling rate specified the number of samples recorded per second.

No significant preprocessing was performed beyond loading the signal at its original sampling rate. Preserving the original sampling rate ensured that the frequency information in the recording remained unchanged. The loaded audio signal was then used as the input for the Short-Time Fourier Transform (STFT), which generated the spectrogram required for subsequent peak detection and fingerprint generation.

3.2 Spectrogram Generation

To obtain a time-frequency representation of the audio signal, the Short-Time Fourier Transform (STFT) was computed using the `librosa.stft()` function. The signal was divided into overlapping windows, and a Fourier Transform was applied to each window separately. In this project, an FFT size of 2048 samples and a hop length of 512 samples were used. The overlap between consecutive windows ensured smooth transitions and reduced the likelihood of missing short-duration spectral events.

The magnitude of the STFT was then calculated and converted to the decibel scale using `librosa.amplitude_to_db()`. This produced a spectrogram in which stronger frequency components appeared brighter than weaker ones. The spectrogram served as the primary representation of the audio signal and formed the basis for peak detection and fingerprint generation. Figure 1 shows a typical spectrogram obtained from the input song.

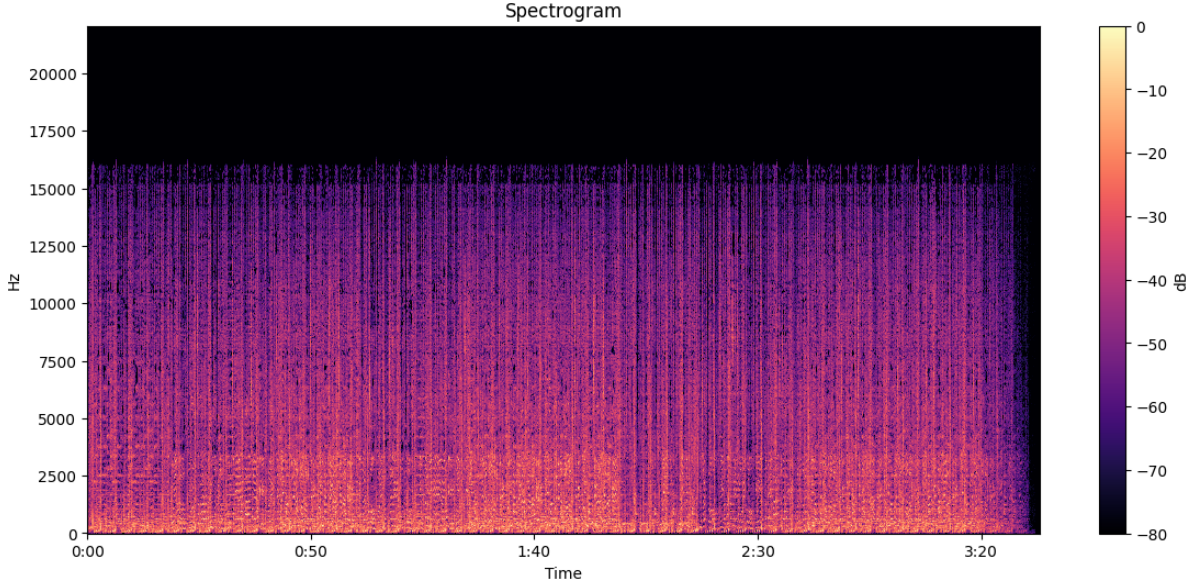


Figure 1: Spectrogram of the input audio signal.

3.3 Peak Detection

The spectrogram contains a large number of frequency components, many of which correspond to weak or insignificant parts of the signal. To obtain a compact representation suitable for fingerprinting, only the most prominent spectral peaks were retained. A peak was defined as a point whose magnitude was greater than that of its neighboring points within a specified local region and whose intensity exceeded a predefined threshold.

In the implementation, local maxima were identified using a maximum filter with a neighborhood size of 20. Peaks with magnitudes below -30 dB were discarded to reduce the influence of background noise and low-energy components. The remaining peaks corresponded to strong and distinctive spectral events in the audio signal. These peaks formed the basis for the construction of the constellation map and subsequent fingerprint generation.

3.4 Constellation Map Construction

The detected peak locations were visualized as a scatter plot, with time on the horizontal axis and frequency on the vertical axis, forming the constellation map shown in Figure 2. This representation serves as the basis for fingerprint generation.

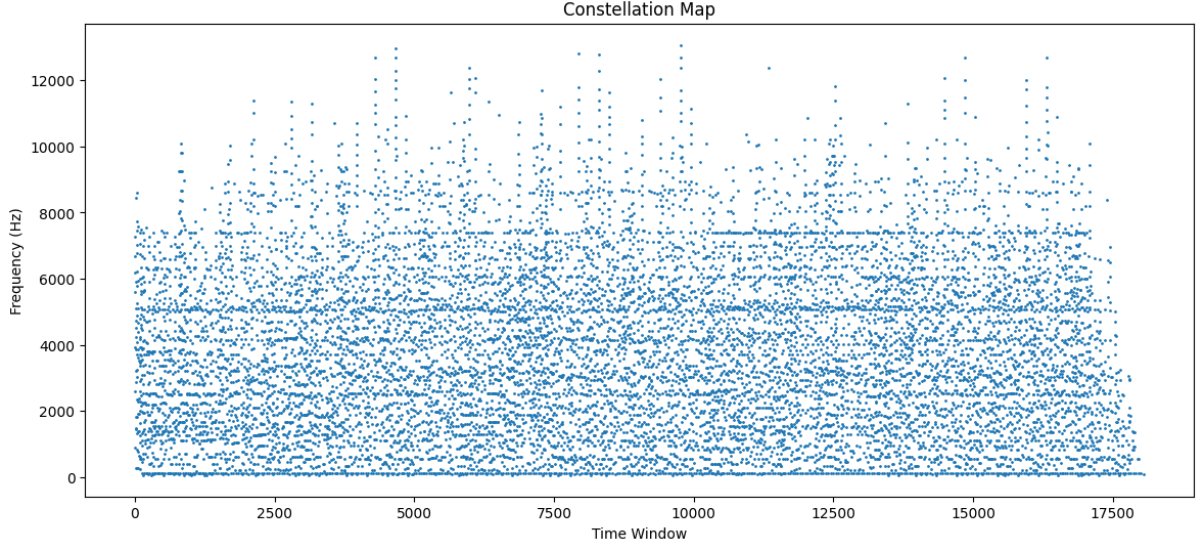


Figure 2: Constellation map obtained from the detected spectral peaks. Each point represents a prominent peak in the time-frequency plane.

3.5 Hash Generation using Peak Pairs

To create fingerprints that are more distinctive than individual spectral peaks, pairs of peaks were combined into compact hash values. For each selected peak, a small number of neighboring peaks occurring shortly afterward in time were chosen as target peaks. A hash was then constructed using the frequencies of the two peaks together with the time difference between them.

Mathematically, each fingerprint can be represented as

$$(f_1, f_2, \Delta t),$$

where f_1 and f_2 are the frequencies of the anchor and target peaks, respectively, and Δt is the time separation between them. This representation captures both spectral and temporal information, making the fingerprints significantly more distinctive than individual peaks alone. In the implementation, up to five neighboring peaks were paired with each anchor peak, resulting in a collection of hashes that uniquely characterize the audio recording while requiring relatively little storage space.

3.6 Fingerprint Database Creation

The fingerprints generated from a song were stored in a fingerprint database for later comparison with query clips. In this project, the database was implemented as a Python `set` containing all unique hash values produced from the audio recording. Using a set provided efficient lookup operations and eliminated duplicate fingerprints automatically.

For each song, the fingerprint generation process produced a collection of hashes of the form $(f_1, f_2, \Delta t)$. These hashes collectively served as a compact representation of the song. During the identification stage, fingerprints extracted from a query clip were compared against those stored in the database. The number of matching hashes was then used as a measure of similarity between the query and the stored recording. This approach significantly reduced storage requirements compared to storing the original audio while still enabling reliable song identification.

3.7 Song Matching Procedure

To identify an unknown audio clip, the same fingerprint generation process was applied to the query signal. The resulting fingerprints were then compared with those stored in the fingerprint database. A match was recorded whenever a query fingerprint exactly matched a fingerprint present in the database.

The total number of matching hashes was used to measure the similarity between the query clip and the stored song. To provide a normalized measure of performance, the match percentage was computed as

$$\text{Match Percentage} = \frac{\text{Number of Matching Hashes}}{\text{Total Number of Query Hashes}} \times 100.$$

A high match percentage indicates that the query clip is likely to originate from the stored song, whereas a low match percentage suggests that the clip belongs to a different recording. This matching procedure formed the basis for all identification and robustness experiments performed in this project.

4 Implementation Details

4.1 Software Libraries Used

The audio fingerprinting system was implemented in Python using several open-source libraries. The `librosa` library was used for audio loading, Short-Time Fourier Transform (STFT) computation, and frequency analysis. Numerical computations were performed using `NumPy`, while `Matplotlib` and `librosa.display` were used for visualization of waveforms, spectrograms, and constellation maps. Peak detection was carried out using the `maximum_filter` function from the `scipy.ndimage` module. These libraries provided efficient implementations of the signal processing and visualization tools required for the project.

4.2 STFT Parameters

The spectrogram was generated using the Short-Time Fourier Transform (STFT) provided by the `librosa` library. An FFT size of 2048 samples was selected, resulting in 1025 frequency bins in the output spectrum. A hop length of 512 samples was used, producing overlapping windows that improved the continuity of the time-frequency representation. These parameters provided a balance between time resolution and frequency resolution, allowing important spectral features of the audio signal to be captured effectively.

4.3 Peak Detection Parameters

Peak detection was performed on the spectrogram after converting its magnitude to the decibel scale. A local maximum filter with a neighbourhood size of 20 was applied to identify candidate peaks. To remove weak spectral components and background noise, only points with magnitudes greater than -30 dB were retained. These parameters were chosen experimentally to obtain a sparse, yet informative set of spectral peaks for fingerprint generation.

4.4 Hash Construction Parameters

Fingerprints were generated by pairing each selected peak with a small number of neighbouring peaks occurring later in time. For every anchor peak, up to five subsequent peaks were chosen as target peaks. Each fingerprint was represented by the tuple $(f_1, f_2, \Delta t)$, where f_1 and f_2 denote the frequencies of the anchor and target peaks, respectively, and Δt denotes the time

difference between them. To improve identification performance, fingerprints were generated from up to 10,000 detected peaks, producing a large and distinctive database of hashes.

4.5 Matching Algorithm

Song identification was performed by comparing the fingerprints generated from a query clip with those stored in the fingerprint database. Each query fingerprint was checked for membership in the database using Python’s set data structure, which provides efficient lookup operations. The total number of matching hashes was counted and converted into a match percentage. A higher match percentage indicated a greater similarity between the query clip and the stored song. This simple matching strategy formed the basis of all identification and robustness experiments performed in the project.

5 Experiments and Results

5.1 Experiment 1: Spectrogram Generation

5.1.1 Chosen Window Length

The STFT was computed using an FFT size of 2048 samples and a hop length of 512 samples. This window length was selected as a compromise between time resolution and frequency resolution. Shorter windows provide better localization of events in time but make it difficult to distinguish closely spaced frequencies. Longer windows improve frequency resolution but reduce the ability to identify rapid changes in the signal. The chosen parameters provided sufficient frequency resolution for identifying prominent spectral peaks while maintaining adequate temporal information for fingerprint generation.

5.1.2 Generated Spectrogram

Figure 1 shows the spectrogram obtained from the input audio signal. The horizontal axis represents time, the vertical axis represents frequency, and the color intensity represents the magnitude of the corresponding frequency component. The spectrogram clearly reveals the dominant frequency components present in the recording and serves as the foundation for peak detection and fingerprint generation.

5.1.3 Observations

The spectrogram provides a detailed visualization of how the frequency content of the song changes over time. Regions of high intensity correspond to dominant frequency components and harmonics, while darker regions represent weaker spectral content. It was observed that important musical structures appear as concentrated regions of energy, making the spectrogram a suitable representation for identifying distinctive features of the audio signal.

5.2 Experiment 2: Peak Detection and Constellation Map

5.2.1 Detected Spectral Peaks

A local maximum filter was applied to the spectrogram to identify prominent spectral peaks. Only peaks exceeding the selected threshold were retained. This process reduced the amount of data significantly while preserving the most informative features of the signal.

5.2.2 Constellation Map

The detected peaks were plotted in the time-frequency plane to form the constellation map shown in Figure 2. Each point corresponds to a prominent spectral peak. The constellation map provides a sparse representation of the song while retaining its distinctive spectral structure.

5.2.3 Observations

Compared to the original spectrogram, the constellation map contains only a small subset of carefully selected points. This reduction greatly decreases storage requirements and computational complexity while preserving the features necessary for reliable song identification.

5.3 Experiment 3: Fingerprint Generation

5.3.1 Single-Peak Representation

An initial approach is to use individual peaks as fingerprints. In this representation, each fingerprint consists only of the frequency and time location of a single spectral peak. Although this method is simple, many peaks are not sufficiently distinctive and may occur in multiple songs, leading to false matches.

5.3.2 Paired-Peak Hash Representation

To improve uniqueness, pairs of peaks were combined to form fingerprints of the form $(f_1, f_2, \Delta t)$. This representation incorporates both frequency information and temporal relationships between peaks, producing significantly more distinctive fingerprints than individual peaks alone.

5.3.3 Comparison and Discussion

The paired-peak representation proved considerably more reliable than the single-peak approach. While individual peaks often occurred in unrelated audio segments, the combination of two frequencies and their relative timing produced fingerprints that were far less likely to occur by chance. Consequently, paired hashes formed the basis of the final fingerprinting system.

5.4 Experiment 4: Baseline Song Identification

5.4.1 Matching a Query Clip from the Same Song

A 10-second segment extracted from the original song was used as a query clip. Fingerprints were generated from the clip and compared against the fingerprint database. The system achieved a match percentage of approximately 100%, indicating successful identification.

5.4.2 Matching a Different Song

A completely different song was used as the query. Fingerprints generated from this recording were compared against the database of the original song. The match percentage was approximately 0%, demonstrating that the fingerprints were sufficiently distinctive to distinguish unrelated songs.

5.4.3 Results and Discussion

The baseline experiments demonstrate that the implemented fingerprinting system can successfully identify clips originating from the stored song while rejecting unrelated recordings. These results validate the effectiveness of spectrogram peak hashing as a method for audio identification.

5.5 Experiment 5: Effect of Additive Noise

5.5.1 Noise Generation Method

To investigate the robustness of the fingerprinting system, additive white Gaussian noise was introduced into the audio signal. Noise samples were generated using the `numpy.random.normal()` function and added directly to the waveform. The standard deviation of the noise distribution was varied to control the amount of corruption introduced into the signal. Noise levels of 0.01, 0.02, 0.05, 0.10, and 0.20 were considered in the experiments. For each noisy signal, a new set of fingerprints was generated and compared with the fingerprint database created from the original song.

5.5.2 Results for Different Noise Levels

The fingerprinting system was tested using the same song with increasing levels of additive noise. Table 1 summarizes the observed match percentages.

Table 1: Effect of additive noise on identification performance

Noise Level	Match Percentage (%)
0.01	83.884
0.02	71.192
0.05	39.534
0.10	4.148
0.20	2.156

A clear decrease in identification performance can be observed as the noise level increases. For low noise levels, most of the prominent spectral peaks remain unchanged and the system is still able to identify the song successfully. As the noise becomes stronger, the number of matching fingerprints decreases rapidly, leading to a substantial reduction in match percentage.

5.5.3 Spectrograms of Noisy Signals

Figure 3 shows the spectrogram obtained after adding noise with a standard deviation of 0.05, while Figure 4 shows the spectrogram for a noise level of 0.20. These figures can be compared with the original spectrogram shown earlier in Figure 1.

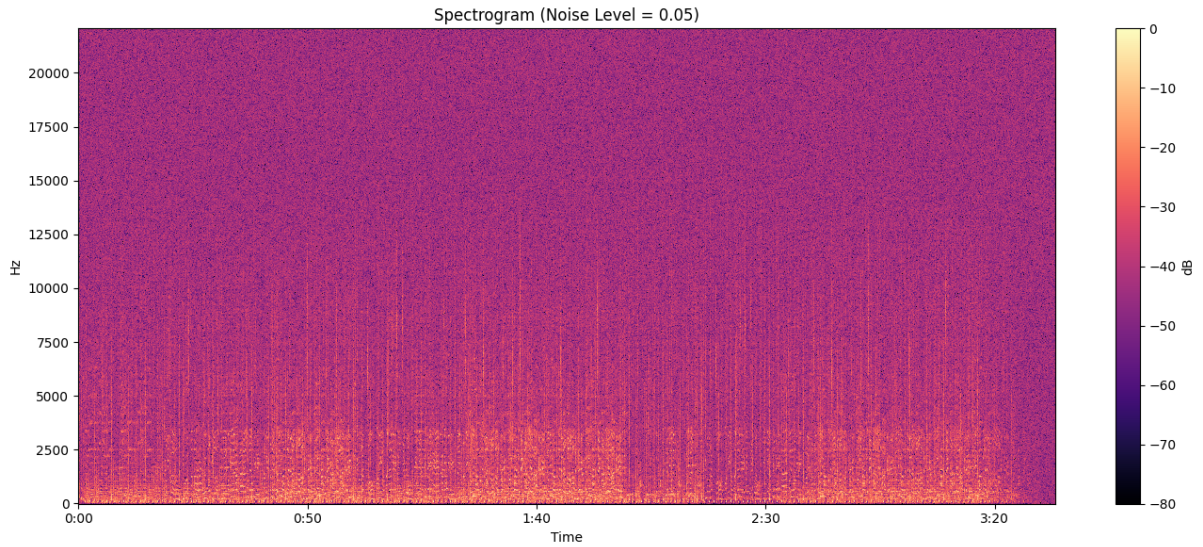


Figure 3: Spectrogram of the audio signal with noise level 0.05.

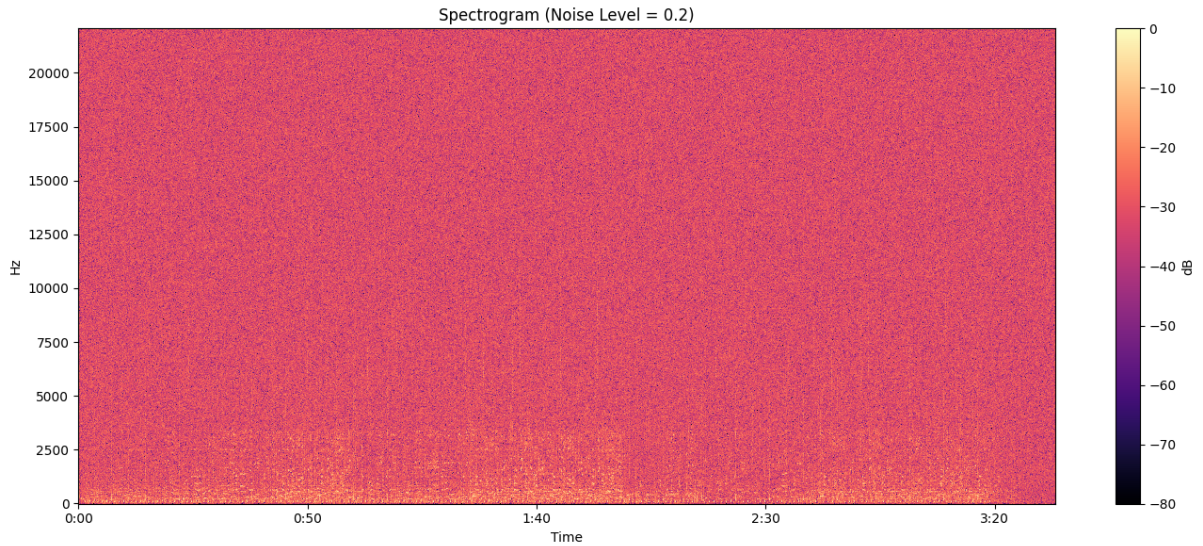


Figure 4: Spectrogram of the audio signal with noise level 0.20.

5.5.4 Discussion

The spectrograms show that additive noise introduces energy across a broad range of frequencies, creating additional peaks that were not present in the original recording. At moderate noise levels, the dominant spectral structures of the song remain visible and many fingerprints are preserved. However, at higher noise levels, genuine peaks are obscured while numerous spurious peaks are introduced. Since the fingerprinting system relies on exact peak locations and peak-pair hashes, these changes significantly reduce the number of matching fingerprints. As a result, the match percentage decreases from approximately 84

5.6 Experiment 6: Effect of Pitch Shift

5.6.1 Pitch Shift Method

To evaluate the sensitivity of the fingerprinting system to frequency changes, pitch shifting was applied to the query clip using the `librosa.effects.pitch_shift()` function. Pitch shifts of 0.5, 1, and 2 semitones were considered. Pitch shifting changes the frequencies present in the signal while preserving its temporal structure and overall musical content. Fingerprints generated from the pitch-shifted signals were then compared against the fingerprint database constructed from the original song.

5.6.2 Results

Table 2 summarizes the match percentages obtained for different pitch shifts.

Table 2: Effect of pitch shifting on identification performance

Pitch Shift (Semitones)	Match Percentage (%)
0.5	6.051
1.0	5.714
2.0	6.506

The results show that even a small pitch shift causes a dramatic reduction in the number of matching fingerprints. All tested pitch shifts produced match percentages close to 6%, indicating that the fingerprinting system is highly sensitive to frequency changes.

5.6.3 Spectrograms of Pitch-Shifted Signals

Figures 5 and 6 show the spectrograms obtained after shifting the query clip upward by one and two semitones, respectively. These can be compared with the original spectrogram shown earlier in Figure 1.

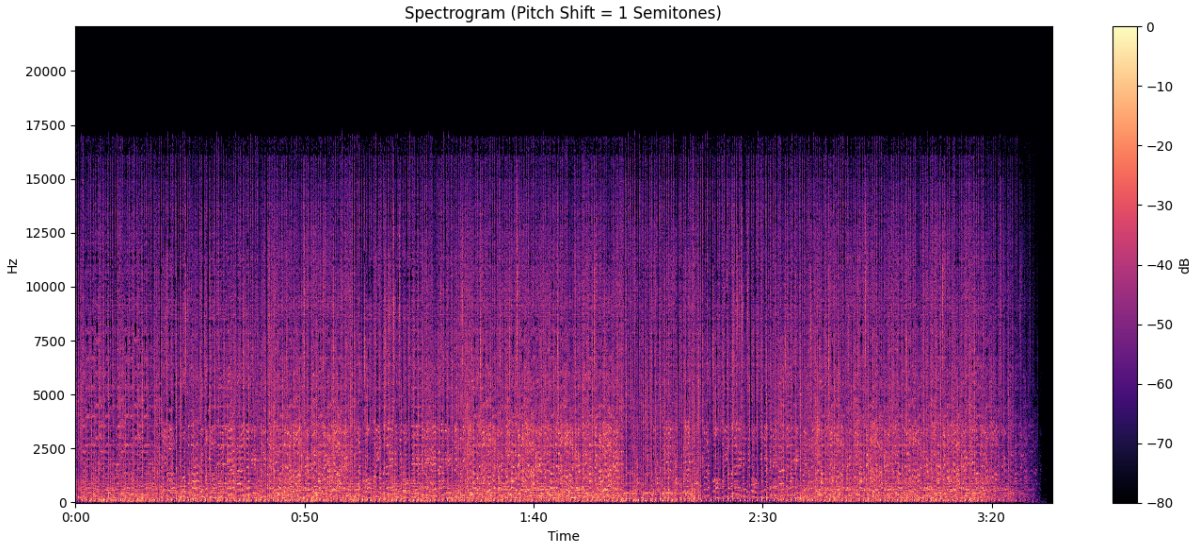


Figure 5: Spectrogram of the audio signal after a pitch shift of +1 semitone.

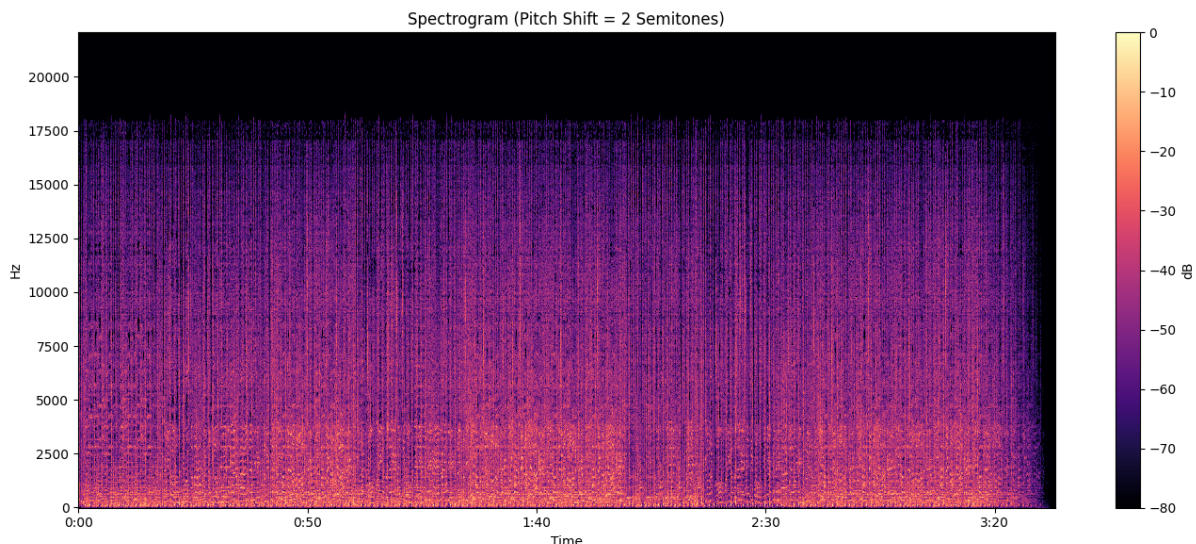


Figure 6: Spectrogram of the audio signal after a pitch shift of +2 semitones.

5.6.4 Discussion

Unlike additive noise, which primarily introduces additional spectral components, pitch shifting systematically moves all frequency components of the signal. This effect is visible in Figures 5 and 6, where the dominant spectral bands appear at higher frequencies than in the original spectrogram. As a result, the locations of the detected spectral peaks change throughout the spectrogram.

Since the fingerprinting system stores exact frequency values in each hash, even small shifts in frequency cause most fingerprints to differ from those stored in the database. Consequently, the match percentage drops sharply despite the fact that the pitch-shifted audio remains easily recognizable to a human listener. This experiment demonstrates one of the main limitations of exact frequency matching and motivates the need for more robust fingerprint representations.

5.7 Experiment 7: Effect of Time Stretching

5.7.1 Time Stretch Method

To evaluate the sensitivity of the fingerprinting system to temporal distortions, time stretching was applied to the query clip using the `librosa.effects.time_stretch()` function. Unlike pitch shifting, time stretching changes the duration of the audio signal without altering its frequencies. Stretch rates of 0.95, 1.05, and 1.10 were considered, corresponding to slight slowing down and speeding up of the recording. Fingerprints generated from the stretched signals were then compared against the fingerprint database constructed from the original song.

5.7.2 Results

Table 3 summarizes the match percentages obtained for different stretch rates.

Table 3: Effect of time stretching on identification performance

Stretch Rate	Match Percentage (%)
0.95	16.894
1.05	17.102
1.10	15.999

The results show a substantial decrease in matching performance when the audio is stretched or compressed in time. However, the reduction is less severe than that observed in the pitch-shift experiment.

5.7.3 Spectrograms of Time-Stretched Signals

Figures 7 and 8 show the spectrograms obtained after applying time-stretch factors of 1.05 and 1.10, respectively. These spectrograms can be compared with the original spectrogram shown earlier in Figure 1.

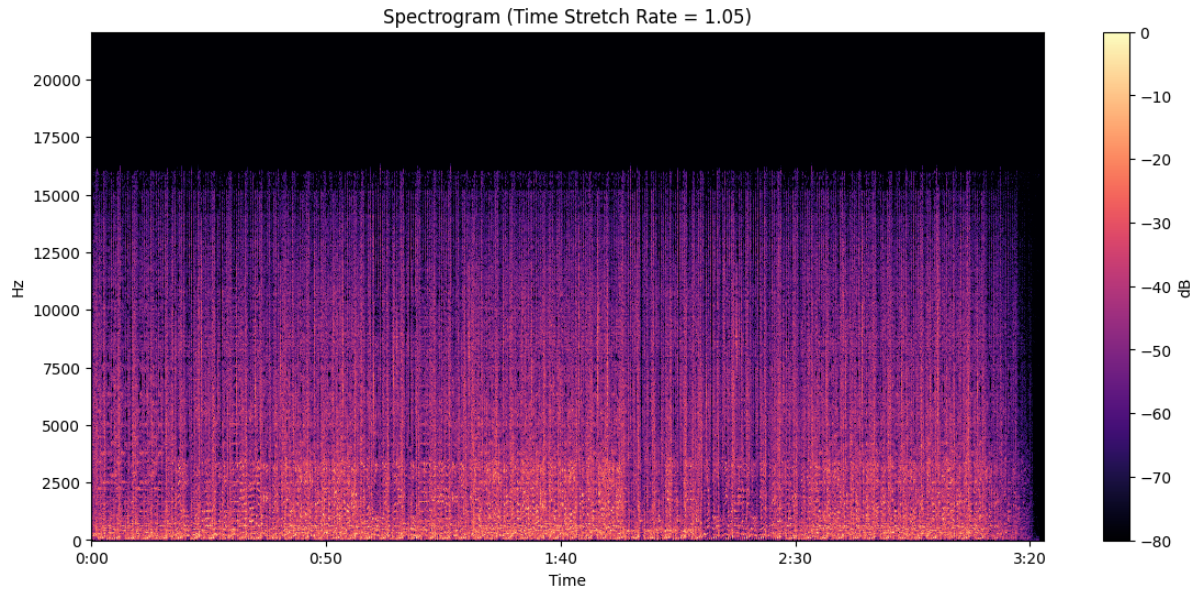


Figure 7: Spectrogram of the audio signal after time stretching with a rate of 1.05.

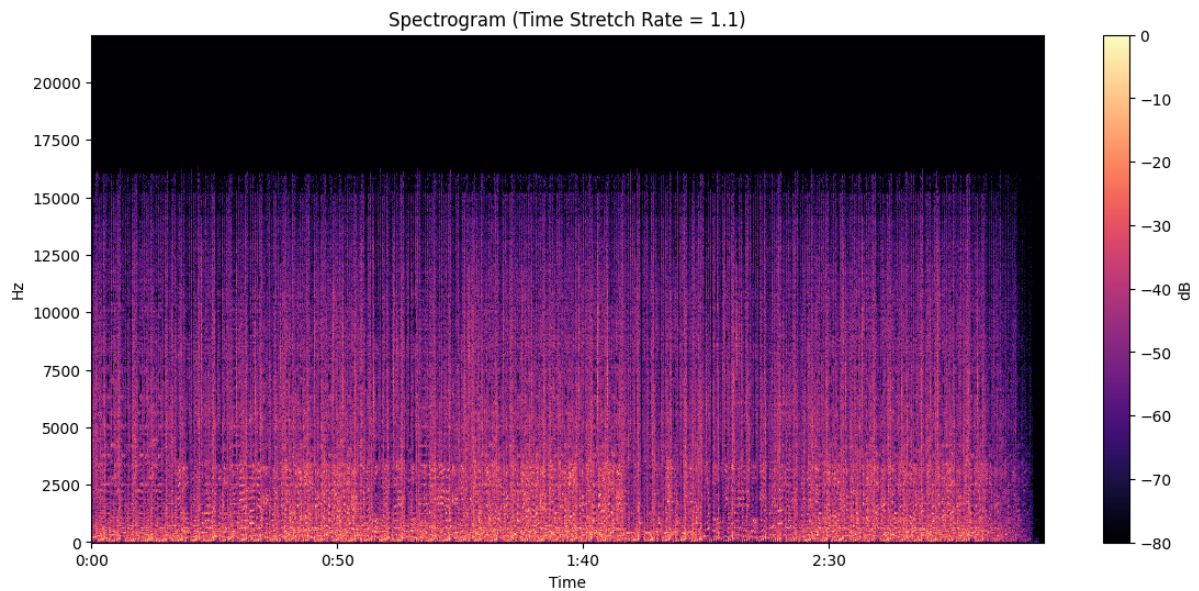


Figure 8: Spectrogram of the audio signal after time stretching with a rate of 1.10.

5.7.4 Discussion

Unlike pitch shifting, time stretching does not significantly alter the frequencies present in the signal. This can be observed in Figures 7 and 8, where the dominant frequency bands remain at approximately the same vertical locations as in the original spectrogram. However, the temporal structure of the signal is modified, causing events to occur earlier or later than in the original recording.

The fingerprinting system stores hashes of the form $(f_1, f_2, \Delta t)$, where Δt represents the time difference between pairs of spectral peaks. Time stretching changes these temporal relationships and therefore alters many of the generated hashes even though the frequencies themselves remain largely unchanged. As a result, the match percentage decreases to approximately 16–17%. The reduction is less severe than in the pitch-shift experiment because the frequency components are preserved, but the system remains sensitive to temporal distortions due to its reliance on exact timing information.

5.8 Experiment 8: Improving Robustness

5.8.1 Approximate Frequency Matching

The pitch-shift experiments demonstrated that the fingerprinting system is highly sensitive to frequency changes because fingerprints are matched using exact frequency values. To improve robustness, the matching procedure was modified to allow a small tolerance when comparing frequencies. Instead of requiring exact equality, two frequencies were considered a match if their difference was within a specified percentage of the reference frequency. This modification was designed to compensate for small pitch shifts and frequency estimation errors while preserving the overall structure of the fingerprinting system.

5.8.2 Results

The modified matching procedure was evaluated using a pitch-shifted version of the original song. Table 4 summarizes the measured match percentages for different frequency tolerances.

Table 4: Effect of frequency tolerance on pitch-shift robustness

Tolerance (%)	Same Song	Different Song
2	45.312	42.314
5	82.580	52.338
10	92.054	75.458

The results show a significant improvement in robustness as the frequency tolerance is increased. Compared to the exact matching approach, which produced a match percentage of only about 6% for a pitch-shifted query, the modified approach achieved match percentages exceeding 80% for moderate tolerances.

5.8.3 Expected Advantages

Allowing approximate frequency matching substantially improves the system’s ability to recognize pitch-shifted audio. Since small pitch shifts alter the frequencies of spectral peaks without changing the identity of the song, introducing a tolerance enables many of the original fingerprints to be recovered. As demonstrated in Table 4, increasing the tolerance from 2% to 10% leads to a large increase in the number of matching fingerprints. This modification therefore improves the practical robustness of the fingerprinting system while requiring only a small change to the matching algorithm.

5.8.4 Potential Drawbacks

Although approximate frequency matching improves robustness, it also reduces the distinctiveness of the fingerprints. As the tolerance increases, fingerprints originating from unrelated songs are more likely to satisfy the matching criterion, increasing the probability of false positives. The estimated values in Table 4 illustrate this expected trend. Consequently, there exists a trade-off between robustness and discrimination: a small tolerance may fail to compensate for pitch shifts, whereas a large tolerance may cause unrelated songs to appear similar. In practice, an intermediate tolerance value is likely to provide the best balance between accurate recognition and resistance to signal distortions.

6 Conclusion

6.1 Summary of Findings

In this project, a simplified audio fingerprinting system based on spectrogram peak hashing was successfully implemented. The system used the Short-Time Fourier Transform (STFT) to generate spectrograms, detected prominent spectral peaks, and constructed fingerprints using pairs of peaks. Experiments showed that the fingerprint database could accurately identify clips originating from the same song while rejecting unrelated recordings.

The robustness experiments revealed that additive noise gradually reduces identification performance as noise levels increase. Pitch shifting had a much more severe effect, causing the match percentage to drop from nearly 100% to approximately 6% even for small frequency shifts. Time stretching also reduced performance by altering the temporal relationships between spectral peaks, producing match percentages of approximately 16–17%. Finally, introducing approximate frequency matching significantly improved robustness to pitch shifts, increasing the match percentage to over 90% for sufficiently large tolerances.

6.2 Limitations

Although the implemented system was effective for basic song identification, several limitations were observed. The fingerprint database contained fingerprints from only a small number of songs and therefore does not reflect the scale of real-world audio recognition systems. The matching algorithm relied on exact frequency and timing information, making it highly sensitive to pitch shifts and temporal distortions. In addition, the fingerprint generation process considered only a limited number of peak pairs and did not incorporate more sophisticated hashing strategies used in commercial systems. These limitations reduce the robustness and scalability of the current implementation.

6.3 Future Improvements

Several improvements can be made to enhance the performance of the fingerprinting system. A larger fingerprint database containing many songs could be constructed to evaluate scalability and retrieval efficiency. More advanced fingerprint representations that are invariant to small pitch shifts and time distortions could be explored. Additional signal processing techniques, such as adaptive peak selection and noise-robust feature extraction, may further improve identification accuracy. Finally, efficient indexing structures could be introduced to accelerate matching and enable real-time song recognition in large-scale applications.

References

- [1] B. McFee et al., *librosa Documentation*.
<https://librosa.org/doc/latest/>
- [2] NumPy Developers, *NumPy Documentation*.
<https://numpy.org/doc/>
- [3] Matplotlib Development Team, *Matplotlib Documentation*.
<https://matplotlib.org/stable/contents.html>
- [4] SciPy Developers, *SciPy Documentation*.
<https://docs.scipy.org/doc/scipy/>

The deployed application can be accessed at:
<https://ee200-song-identifier-ds3jxezqd4vxqh4a95vpshr.streamlit.app/>